

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE CODE: CSA1417

COURSE NAME: Compiler Design

CO1: Apply lexical analysis for token specification and recognition using LEX tool (BL3)

Assessment Tool Weightage: 40%

Total Marks: 50

Code of Conduct

I K.Manashvitha(1925260260) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: K.Manashvitha

ANALYTIC PROBLEM SOLVING:

1. Trace Lexical and Syntax Analysis

Statement: result = (a + b) * c - 10;

1.1 Lexical Analysis – Lexemes and Tokens

The lexical analyzer scans the character stream and produces the following lexemes and corresponding tokens:

Lexeme	Token
result	id (identifier)
=	assign_op
(	lparen
a	id (identifier)
+	add_op
b	id (identifier)
)	rparen
*	mul_op
c	id (identifier)
-	sub_op
10	num (integer constant)

;	semicolon
---	-----------

Classification

Operands / Identifiers: result, a, b, c

Operators: =, +, *, –

Delimiters / Separators: (,), ; **Constant:**

10

1.2 Syntax Analysis – Parse Tree & Operator Precedence

The syntax analyzer (parser) receives the token stream and constructs a parse tree / Abstract Syntax Tree (AST) according to the grammar of expressions. Operator precedence and associativity are applied as follows (highest to lowest):

1. Parentheses () – highest precedence
2. *, / – left-associative
3. +, – – left-associative
4. Assignment = – right-associative

The parentheses force (a + b) to be evaluated first. Multiplication has higher precedence than subtraction, so (a + b) * c is computed before subtracting 10.

Abstract Syntax Tree (simplified):

```

=
/\
result -
    /\
    * 10
    /\
    + c
    /\ a b

```

1.3 Role of Semantic Analysis

- Checks that identifiers result, a, b, and c are declared in the symbol table.
- Verifies type compatibility – all operands should be numeric types for the arithmetic operations.
- Ensures the assignment is type-safe (RHS type is compatible with LHS type).
- May insert implicit type conversions if needed (e.g., integer to float).

2. Analyze Lexical Ambiguity and Longest-Match Rule

Statement: value = a >= b && c != d;

2.1 Possible Ambiguities

The following character sequences can cause ambiguity during scanning:

- > vs >=
- ! vs !=
- & vs &&

If the lexer used a shortest-match rule, the expression a >= b could be incorrectly tokenized as a, >, =, b instead of a, >=, b.

2.2 Longest-Match (Maximal Munch) Rule

The lexical analyzer always prefers the longest possible valid lexeme:

- When it sees '>', it looks ahead. Finding '=', it matches '>=' as a single relational operator.
- When it sees '!', it looks ahead. Finding '=', it matches '!='.
- When it sees '&', it looks ahead. Finding another '&', it matches '&&'.

2.3 Suitable Token Specifications (Regular Expressions)

Token	Regular Expression
id	letter (letter digit)*
relop	< <= > >= == !=
logand	&&
logor	
assign	=

2.4 Order of Pattern Specification

Patterns that can overlap must be ordered carefully (combined with longest-match priority):

1. Multi-character operators first ($\geq$, $\leq$, $==$, $!=$, $\&\&$, $||$)
2. Single-character operators next ($>$, $<$, $!$, $=$, $\&$, $|$)
3. Identifiers and keywords last

3. Determine Token Stream and Handle Lexical Errors

Program fragment:

```
int total = 250; float  
average = total / 5.0;  
result@ = average + 10;
```

3.1 Line-by-Line Token Stream

Line 1: int total = 250;

Lexeme	Token Type
int	keyword
total	identifier
=	operator
250	integer constant
;	separator

Line 2: float average = total / 5.0;

Lexeme	Token Type
float	keyword

average	identifier
=	operator
total	identifier
/	operator
5.0	floating constant
;	separator

Line 3: result@ = average + 10;

Lexeme	Token Type
result	identifier
@	LEXICAL ERROR (illegal character)
=	operator
average	identifier
+	operator
10	integer constant
;	separator

3.2 Handling the Lexical Error

The character '@' is not part of any valid token pattern in the language.

A typical lexical analyzer handles it as follows:

1. Reports an error message: "Illegal character '@' at line 3".
2. Skips / discards the illegal character (panic-mode recovery).
3. Continues scanning from the next character ('=').

Token stream after recovery continues normally with: =, average, +, 10, ;

4. Design Regular Expressions for Token Recognition

4.1 Regular Expressions for Token Categories

Token Category	Regular Expression
Keyword	int float while return
Identifier	[a-zA-Z][a-zA-Z0-9]*
Integer constant	[0-9]+
Floating-point constant	[0-9]+\.[0-9]+
Assignment	=
Relational operators	< <= > >= ==

Arithmetic operators	+ - * /
Separators	() { } ;

Note: Keywords are recognized by first matching the identifier pattern and then looking up the string in a keyword table (or by giving keyword patterns higher priority).

4.2 Complete Token Stream

Input program:

```
float total = price * 12.5;
while (total >= 100) {
total = total - 10;
} return
total;
```

Lexeme	Token
float	keyword
total	identifier
=	assign
price	identifier
*	mul_op
12.5	float_const
;	semicolon
while	keyword
(	lparen
total	identifier
>=	relop
100	int_const
)	rparen
{	lbrace
total	identifier
=	assign
total	identifier

-	sub_op
10	int_const
;	semicolon
}	rbrace
return	keyword
total	identifier
;	semicolon

4.3 Distinguishing Keywords and Floating Constants

- Keyword “float”: It matches the identifier pattern $[a-zA-Z][a-zA-Z0-9]^*$. After matching, the lexer looks up the string in a keyword table (or keyword patterns have higher priority) and classifies it as a keyword rather than an ordinary identifier.
- Floating constant “12.5”: It matches the floating-point pattern $[0-9]^+\.[0-9]^+$. The presence of the decimal point prevents it from being recognized as an integer constant $[0-9]^+$.

5. Analyze the Complete Compiler Processing

Statement: `int average = total / count;`

5.1 Phase-by-Phase Analysis

Phase	Input	Output / Action
Lexical Analysis	Character stream	Tokens: keyword(int), id(average), assign, id(total), div, id(count), semicolon
Syntax Analysis	Token stream	Parse tree / AST representing a declaration + assignment expression
Semantic Analysis	AST + Symbol Table	Checks that total & count are declared; type checking of division & assignment; may insert type conversions
Intermediate Code Generation	Annotated AST	Three-address code: <code>t1 = total / count ; average = t1</code>
Code Generation	Intermediate representation	Target instructions (example): <code>LOAD R1, total DIV R1, count STORE average, R1</code>
Symbol Table	Used in all phases	Stores names, types, scope and memory offsets of average, total and count

5.2 Front End / Middle End / Back End Organization

Front End (Lexical + Syntax + Semantic Analysis)

Language-dependent. Produces an intermediate representation and fills the symbol table. Independent of the target machine.

Middle End (Optimizer – optional)

Machine-independent optimizations on the intermediate code (constant folding, common sub-expression elimination, etc.).

Back End (Code Generation + Machine-dependent optimizations)

Target-machine specific. Generates assembly or machine code using the intermediate representation and symbol-table information (offsets, register allocation).

Benefit of this organization:

This separation allows the same front end to be reused for multiple target architectures and simplifies maintenance and portability of the compiler.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem

Method Selection	20%	Selects most appropriate method/for mula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method
Solution Process	25%	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process
Accuracy of Calculation	20%	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations
Interpretation of Results	15%	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited unclear or interpretation	No interpretation